

Practical Lab Manual

Deep Learning (410251)

For Final Year Computer Engineering (2019 Course)



Student Name: _____

Seat No / Roll No: _____ Class: _____

Branch: _____



Department of Computer Engineering

Al Jamia Mohammediyah Education Society's
Maulana Mukhtar Ahmad Nadvi Technical Campus
Mansoor Campus, Malegaon (Nashik)

Certificate

This is to certify that.....

Roll No:Exam Seat No:..... Class:

Branch.....has Successfully Completed

Deep Learning Lab Experiments and Assignments as per the term work of Savitri-bai

Phule Pune University, Pune.

Subject Teacher

Head of Department

Principal

Maulana Mukhtar Ahmad Nadvi Technical Campus
Department of Computer Engineering

Vision of the Institute

Empowering society through quality education and research for the socio-economic development of the region.

Mission of the Institute

- Inspire students to achieve excellence in science and engineering.
- Commit to making quality education accessible and affordable to serve society.
- Provide transformative, holistic, and value-based immersive learning experiences for students.
- Transform into an institution of global standards that contributes to nation-building.
- Develop sustainable, cost-effective solutions through innovation and research.
- Promote quality education in rural areas.

Vision of the Department

To build strong research and learning environment producing globally competent professionals and innovators who will contribute to the betterment of the society.

Mission of the Department

- To create and sustain an academic environment conducive to the highest level of research and teaching.
- To provide state-of-the-art laboratories which will be up to date with the new developments in the area of computer engineering.
- To organize competitive event, industry interactions and global collaborations in view of providing a nurturing environment for students to prepare for a successful career and the ability to tackle lifelong challenges in global industrial needs.
- To educate students to be socially and ethically responsible citizens in view of national and global development.

Program Outcomes (POs)

Learners are expected to know and be able to

PO1	Engineering knowledge	Apply the knowledge of mathematics, science, Engineering fundamentals, and an Engineering specialization to the solution of complex Engineering problems.
PO2	Problem analysis	Identify, formulate, review research literature and analyze complex Engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and Engineering sciences.
PO3	Design / Development of Solutions	Design solutions for complex Engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and Environmental considerations.
PO4	Conduct Investigations of Complex Problems	Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern Tool Usage	Create, select, and apply appropriate techniques, resources, and modern Engineering and IT tools including prediction and modeling to complex Engineering activities with an understanding of the limitations.
PO6	The Engineer and Society	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	Environment and Sustainability	Understand the impact of the professional Engineering solutions in societal and Environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	Ethics	Apply ethical principles and commit to professional ethics and responsibilities and norms of Engineering practice.
PO9	Individual and Team Work	Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
P10	Communication Skills	Communicate effectively on complex Engineering activities with the Engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions
P11	Project Management and Finance	Demonstrate knowledge and understanding of Engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary Environments.
P12	Life-long Learning	Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSO)

A graduate of the Computer Engineering Program will demonstrate

PSO1

Professional Skills-The ability to understand, analyze and develop computer programs in the areas related to algorithms, system software, multimedia, web design, big data analytics, and networking for efficient design of computer-based systems of varying complexities.

PSO2

Problem-Solving Skills- The ability to apply standard practices and strategies in software project development using open-ended programming environments to deliver a quality product for business success.

PSO3

Successful Career and Entrepreneurship- The ability to employ modern computer languages, environments and platforms in creating innovative career paths to be an entrepreneur and to have a zest for higher studies

Course Objectives:

To implement different deep learning models.

Course Outcomes:

On completion of this course, the students will be able to

CO3: Identify and apply the suitable algorithms to solve AI/ML problems.

CO4: Apply the technique of Deep Neural network for implementing Linear regression and classification.

CO5: Apply the technique of Convolution (CNN) for implementing Deep Learning models.

CO6: Design and develop Recurrent Neural Network (RNN) for prediction.

410251: Course Code : Deep Learning
Any 3 Assignments and 1 Mini Project is Compulsory
Group 1

1.	Linear regression by using Deep Neural network: Implement Boston housing price prediction problem by Linear regression using Deep Neural network. Use Boston House price prediction dataset.
2.	Classification using Deep neural network (Any One from the following) 1. Multiclass classification using Deep Neural Networks: Example: Use the OCR letter recognition dataset https://archive.ics.uci.edu/ml/datasets/letter+recognition 2. Binary classification using Deep Neural Networks Example: Classify movie reviews into positive" reviews and "negative" reviews, just based on the text content of the reviews. Use IMDB dataset
3.	Convolutional neural network (CNN) (Any One from the following) • Use any dataset of plant disease and design a plant disease detection system using CNN. • Use MNIST Fashion Dataset and create a classifier to classify fashion clothing into categories.
4.	Recurrent neural network (RNN) Use the Google stock prices dataset and design a time series analysis and prediction system using RNN.
Group 2	
5.	Mini Project: Human Face Recognition
6.	Mini Project: Gender and Age Detection: predict if a person is a male or female and also their age
7.	Mini Project: Colorizing Old B&W Images: color old black and white images to colorful images

INDEX

Sr No.	Name of Experiment	Date	Marks	Sign
1	Linear regression by using Deep Neural network: Implement Boston housing price prediction Problem by Linear regression using Deep Neural network. Use Boston House price prediction dataset.			
2	Classification using Deep neural network (Any One from the following) 1. Multiclass classification using Deep Neural Networks: Example: Use the OCR letter recognition dataset https://archive.ics.uci.edu/ml/datasets/letter+recognition 2. Binary classification using Deep Neural Networks Example: Classify movie reviews into positive" reviews and "negative" reviews, just based on the text content of the reviews. Use IMDB dataset			
3	Convolutional neural network (CNN) (Any One from the following) - Use any dataset of plant disease and design a plant disease detection system using CNN. - Use MNIST Fashion Dataset and create a classifier to classify fashion clothing into categories			
4	Recurrent neural network (RNN) Use the Google stock prices dataset and design a time series analysis and prediction system using RNN.			

Experiment No - 01

Title: Linear regression using Deep Neural network: Implement Boston housing price prediction problem by Linear regression using Deep Neural network. Use the Boston House price prediction dataset.

Objectives: Students should be able to perform Linear regression by using a Deep Neural network on the Boston House Dataset.

Software Requirement:

Operating system: Windows, Linux, macOS.

Programming Languages: Python, C++, Java.

Theoretical Concept:

What is Linear Regression? Linear regression is a statistical approach that is commonly used to model the relationship between a dependent variable and one or more independent variables. It assumes a linear relationship between variables and uses mathematical methods to estimate the coefficients that best fit the data. Deep neural networks are a type of machine learning algorithm that is modeled after the structure and function of the human brain. They consist of multiple layers of interconnected neurons that process data and learn from it to make predictions or classifications. Linear regression using deep neural networks combines the principles of linear regression with the power of deep learning algorithms. In this approach, the input features are passed through one or more layers of neurons to extract features and then a linear regression model is applied to the output of the last layer to make predictions. The weights and biases of the neural network are adjusted during training to optimize the performance of the model. This approach can be used for a variety of tasks, including predicting numerical values, such as stock prices or housing prices, and classifying data, such as detecting whether an image contains a particular object or not. It is often used in fields such as finance, healthcare, and image recognition.

Example Of Linear Regression:

A suitable example of linear regression using a deep neural network would be predicting the price of a house based on various features such as the size of the house, the number of bedrooms, the location, and the age of the house. In this example, the input features would be fed into a deep neural network, consisting of multiple layers of interconnected neurons. The first few layers of the network would learn to extract features from the input data, such as identifying patterns and correlations between the input features. The output of the last layer would then be passed through a linear regression model, which would use the learned features to predict the price of the house. During training, the weights and biases of the neural network would be adjusted to minimize the difference between the predicted price and the actual price of the house. This process is known as gradient descent, and it involves iteratively adjusting the model's parameters until the optimal values are reached. Once the model is trained, it can be used to predict the price of a new house based on its features. This approach can be used in the real estate industry to provide accurate and reliable estimates of house prices, which can help both buyers and sellers make informed decisions.

A deep neural network is a type of machine learning algorithm that is modeled after the structure and function of the human brain. It consists of multiple layers of interconnected nodes, or artificial neurons, that process data and learn from it to make predictions or classifications. Each layer of the network performs a specific type of processing on the data, such as identifying patterns or correlations between features, and passes the results to the next layer. The layers closest to the input are known as the "input layer", while the layers closest to the output are known as the "output layer". The intermediate layers between the input and output are known as "hidden layers". These layers are responsible for extracting increasingly complex features from the input data and can be deep (i.e., containing many hidden layers) or shallow (i.e., containing only a few hidden layers). Deep neural networks are trained using a process known as backpropagation, which involves adjusting the weights and biases of the nodes based on the error between the predicted output and the actual output. This process is repeated for multiple iterations until the model reaches an optimal level of accuracy. Deep neural networks are used in a variety of applications, such as image and speech recognition, natural language processing, and recommendation systems. They are capable of learning from vast amounts of data. They can automatically extract features from raw data, making them a powerful tool for solving complex problems in a wide range of domains.

How Deep Neural Networks Work

Boston House Price Prediction is a common example used to illustrate how a deep neural network can work for regression tasks. The goal of this task is to predict the price of a house in Boston based on various features such as the number of rooms, crime rate, and accessibility to public transportation. Here's how a deep neural network can work for Boston House Price Prediction:

1. Data Pre-processing: The first step is preprocessing. This involves normalizing the input features to have a mean of 0 and a standard deviation of 1, which helps the network learn more efficiently. The dataset is then split into training and testing sets.

2. Model Architecture: A deep neural network is then defined with multiple layers. The first layer is the input layer, which takes in the normalized features. This is followed by several hidden layers, which can be deep or shallow. The last layer is the output layer, which predicts the house price.

3. Model Training: The model is then trained using the training set. During training, the weights and biases of the nodes are adjusted based on the error between the predicted output and the actual output. This is done using an optimization algorithm such as stochastic gradient descent.

4. Model evaluation: Once the model is trained, it is evaluated using the testing set. The performance of the model is measured using metrics such as mean squared error or mean absolute error.

5. Model prediction: Finally, the trained model can be used to make predictions on new data, such as predicting the price of a new house in Boston based on its features.

By using a deep neural network for Boston House Price Prediction, we can obtain accurate predictions based on a large set of input features. This approach is scalable and can be used for other regression tasks as well.

Boston House Price Prediction Dataset

Boston House Price Prediction is a well-known dataset in machine learning and is often used to demonstrate regression analysis techniques. The dataset contains information about 506 houses in Boston, Massachusetts, USA. The goal is to predict the median value of owner-occupied homes in thousands of dollars.

Implementation:

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 %matplotlib inline
```

```
1 df = pd.read_csv("/content/drive/MyDrive/BostonHousing.csv")
2 df
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

```
1 df.head()
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2

```
1 df.info()
2 df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0   crim        506 non-null    float64
1   zn          506 non-null    float64
2   indus       506 non-null    float64
3   chas        506 non-null    int64
4   nox         506 non-null    float64
5   rm          506 non-null    float64
6   age         506 non-null    float64
7   dis         506 non-null    float64
8   rad         506 non-null    int64
9   tax         506 non-null    int64
10  ptratio     506 non-null    float64
```

```

11 b          506 non-null    float64
12 lstat      506 non-null    float64
13 medv       506 non-null    float64
dtypes: float64(11), int64(3)
memory usage: 55.5 KB

```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	
count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000
mean	3.613524	11.363636	11.136779	0.069170	0.554695	6.284634	68.574901	3.795043	9.549407	408.237154	18.455534	356.674032	12.65
std	8.601545	23.322453	6.860353	0.253994	0.115878	0.702617	28.148861	2.105710	8.707259	168.537116	2.164946	91.294864	7.14
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	2.900000	1.129600	1.000000	187.000000	12.600000	0.320000	1.73
25%	0.082045	0.000000	5.190000	0.000000	0.449000	5.885500	45.025000	2.100175	4.000000	279.000000	17.400000	375.377500	6.95
50%	0.256510	0.000000	9.690000	0.000000	0.538000	6.208500	77.500000	3.207450	5.000000	330.000000	19.050000	391.440000	11.36
75%	3.677083	12.500000	18.100000	0.000000	0.624000	6.623500	94.075000	5.188425	24.000000	666.000000	20.200000	396.225000	16.95
max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000	100.000000	12.126500	24.000000	711.000000	22.000000	396.900000	37.97

```

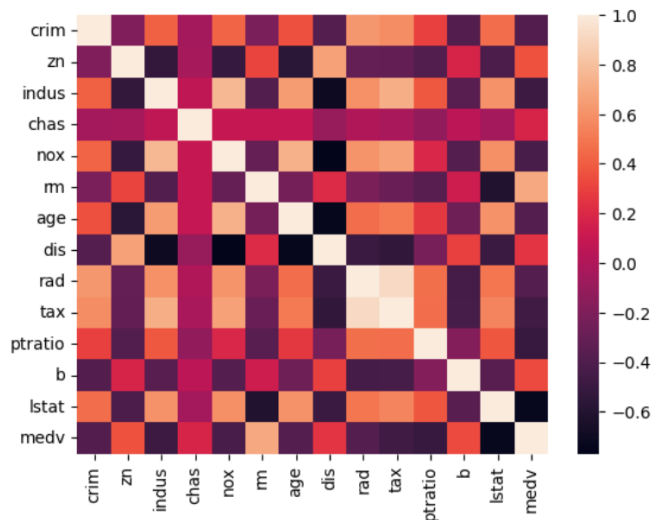
1 X = df.iloc[:, :13]
2 Y = df["medv"]

```

```

1 corr = df.corr()
2 sns.heatmap(corr)
3 plt.show()

```



```

1 x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=.30, random_state=10)

```

```

1 linear_model = LinearRegression()
2 linear_model.fit(x_train, y_train)

```

LinearRegression()

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```

1 predicted_house_price = linear_model.predict(x_test)
2 print("Predicted House Prices")
3 print(predicted_house_price)

```

Predicted House Prices

```

[31.4243217 31.96785487 30.93785448 22.34313349 18.83846235 16.20617519
35.92908162 14.74157477 25.07700756 37.13230282 21.47652971 30.92661826
28.07823424 34.02599249 33.7778476 40.63701192 24.25899783 23.43019291
25.547906 21.34469147 32.65467539 17.80506124 25.46149722 25.0207691
32.51742137 20.51357936 19.47165255 16.87107974 38.44316206 0.3888111
32.39559257 32.15518102 26.05305015 23.82049084 20.56494632 19.66990981
3.53212643 35.21058387 27.03280773 27.67994129 34.36642896 29.82003002
18.31717228 31.55109654 17.93465111 28.4618882 19.39950216 21.60782793
38.10391926 16.45101411 24.51003632 19.57072199 24.53359986 34.34589029
26.74381857 34.86340026 21.02859444 19.77400901 18.68461884 24.64911818
19.89923131 23.52871967 39.56690084 42.81848202 30.38116596 16.96889789
23.83550068 3.18952008 31.52469303 28.66536677 18.40003941 27.1252398
19.56625808 25.28579071 25.04642543 10.30562974 38.97693514 8.17901104
18.51160158 30.32060192 22.85826904 20.98739809 20.03256896 28.70004491
30.83986409 28.22575374 26.26715448 31.56221764 22.16821072 -6.40058282
21.56768106 19.91022274 24.98025707 23.46247865 19.27692546 18.74766072
27.39474857 22.21870161 26.82617865 23.38366235 23.91947259 19.26003338
21.09721048 10.97767848 13.82483416 20.72378817 23.53977916 13.92807616

```

```

28.87070782 15.75931299 15.18847286 22.31904753 26.57865025 28.92895886
24.2980433 18.18585276 16.29471281 17.40018499 15.5361744 21.171008
33.71665888 30.02422884 21.13474092 13.93326869 16.19662736 29.30890971
13.1822158 22.06473155 24.33753588 31.89978763 33.44295992 5.90754764
35.2142013 24.1925307 17.45166483 24.25550318 28.41864261 34.52462088
6.2544935 2.12089952 28.40739835 12.49858357 18.22016397 19.69247114
5.52373677 14.45613717 37.18224665 25.78695839 23.32233462 26.36634945
11.47817153 20.44901468]

```

```

1 predicted_dataframe = pd.DataFrame(
2     {"Predicted_Prices": predicted_house_price, "Actual_Prices": y_test})
3 print("Predicted and Actual Price Data frame")
4 print(predicted_dataframe)

```

Predicted and Actual Price Data frame

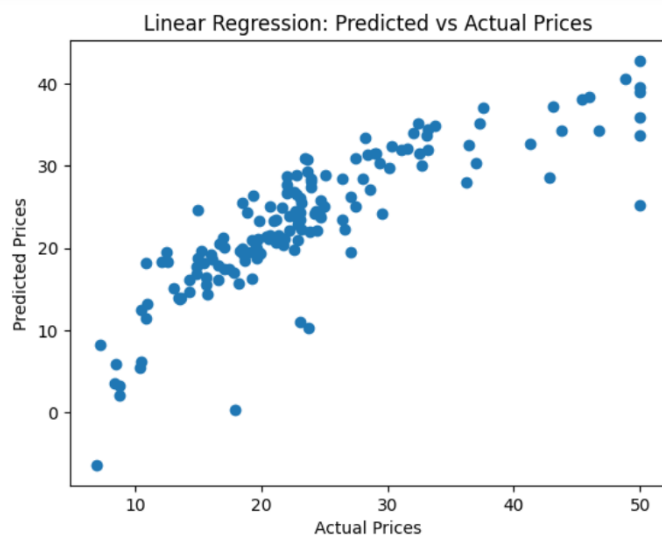
	Predicted_Prices	Actual_Prices
305	31.424322	28.4
193	31.967855	31.1
65	30.937854	23.5
349	22.343133	26.6
151	18.838462	19.6
..	...	...
56	25.786958	24.7
37	23.322335	21.0
66	26.366349	19.4
427	11.478172	10.9
12	20.449015	21.7

[152 rows x 2 columns]

```

1 import matplotlib.pyplot as plt
2
3 # Assuming you have already trained the linear regression model and obtained predictions
4 predicted_house_price = linear_model.predict(x_test)
5
6 # Create a DataFrame with predicted and actual prices
7 predicted_dataframe = pd.DataFrame({"Predicted_Prices": predicted_house_price, "Actual_Prices": y_test})
8
9 # Plotting the linear regression graph
10 plt.scatter(y_test, predicted_house_price)
11 plt.xlabel("Actual Prices")
12 plt.ylabel("Predicted Prices")
13 plt.title("Linear Regression: Predicted vs Actual Prices")
14 plt.show()
15

```



Conclusion: In this way, we can Predict the Boston House Price using a Deep Neural Network.

Experiment No - 02

Title: Binary classification using deep neural networks example: classify movie reviews into "positive" reviews and "negative" reviews, just based on the text content of the reviews. Use the IMDB dataset.

Objectives: The objective of this experiment is to develop a deep neural network model that can accurately classify movie reviews as either "positive" or "negative" based solely on the textual content of the reviews. The model will be trained using the IMDB dataset, which consists of movie reviews along with their corresponding sentiment labels.

Problem Statement: The problem is to create a deep neural network model that can accurately classify movie reviews as positive or negative based solely on the textual content of the reviews. The model will be trained using the IMDB dataset, consisting of labeled movie reviews. The challenge is to design a model architecture and optimize its performance through experimentation with hyperparameters. The trained model will be evaluated on a test set to measure its accuracy and generalization ability. The objective is to develop a reliable and efficient system for automating sentiment analysis of movie reviews using deep learning techniques.

Software Requirement:

- 1. Python:** The experiment will be implemented using the Python programming language, so ensure that Python is installed on your system.
- 2. TensorFlow/Keras:** These deep learning frameworks provide a high-level API for building and training neural networks. Install TensorFlow or Keras to facilitate model development.
- 3. NumPy:** This library is used for numerical computations and array operations. Install NumPy to handle numerical data efficiently.
- 4. IMDB Dataset:** Download the IMDB dataset, which contains movie reviews and their sentiment labels, to train and evaluate the model.
- 5. Text Preprocessing Libraries:** Depending on the specific preprocessing steps required, you may need to install libraries such as NLTK or spaCy for text cleaning, tokenization, and other natural language processing tasks.

Hardware Requirement:

1. CPU/GPU: A reasonably powerful CPU or GPU is recommended for training deep neural networks. While a CPU can handle smaller models, training larger models may benefit from a GPU, which can significantly speed up computations.

2. Memory (RAM): Sufficient memory is required to load and process the dataset, especially if the dataset is large. The exact memory requirement depends on the dataset size and model complexity.

3. Storage: Adequate storage space is necessary to store the IMDb dataset and any intermediate or final model checkpoints, logs, and results.

4. Internet Connection: An internet connection is required to download the IMDb dataset and any necessary software libraries or dependencies.

5. Operating System: The experiment can be conducted on various operating systems such as Windows, macOS, or Linux, depending on your preference and the compatibility of the required software.

Theoretical Concept:

The theoretical concepts underlying the classification of movie reviews using deep neural networks are natural language processing (NLP) and sentiment analysis. Natural Language Processing (NLP) is a branch of artificial intelligence that focuses on understanding and processing human language. It involves techniques for text analysis, tokenization, syntactic and semantic analysis, and sentiment analysis. Sentiment analysis, also known as opinion mining, is a specific task within NLP that aims to determine the sentiment or subjective information expressed in a piece of text. In the context of movie reviews, sentiment analysis involves classifying the reviews into positive or negative sentiments based on the expressed opinions and emotions.

Deep neural networks (DNNs) are a class of machine learning models inspired by the structure and functioning of the human brain. DNNs are particularly effective for NLP tasks as they can learn complex patterns and representations from large amounts of text data. In the case of movie review classification, a deep neural network model is trained using labeled data, where the movie reviews are the input and the sentiment labels (positive or negative) are the target outputs. The model learns to extract meaningful features from the textual content and make

predictions based on those features. Common architectures for text classification include Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs) such as Long Short-Term Memory (LSTM) or Gated Recurrent Unit (GRU), and combinations of these architectures. These models are trained using gradient-based optimization algorithms and loss functions specific to binary classification tasks, such as binary cross-entropy. During training, the model adjusts its internal parameters based on the training data, optimizing the model's ability to classify movie reviews accurately.

The performance of the model is evaluated using metrics such as accuracy, precision, recall, and F1 score on a separate test set. By leveraging deep neural networks and sentiment analysis techniques, it is possible to build powerful models that can automatically classify movie reviews, enabling applications such as sentiment analysis in recommendation systems, market research, and social media monitoring.

How Binary Classification Works:

The process of classifying movie reviews using deep neural networks involves several steps. Here is an overview of how it works:

1. Data Preprocessing

- The IMDb dataset, which contains movie reviews labeled as positive or negative, is loaded.
- The text data undergoes preprocessing steps such as removing HTML tags, and punctuation marks and converting the text to lowercase.
- The reviews are often transformed into a numerical representation suitable for input to the neural network, such as using word embeddings or one-hot encoding.

2. Model Architecture

- A deep neural network model architecture is designed, typically using layers such as embedding layers, convolutional layers, recurrent layers (such as LSTM or GRU), and dense layers.
- The architecture is chosen to effectively capture the semantic meaning and sentiment expressed in the textual content of the reviews.
- The model's architecture and hyperparameters, such as the number of layers, number of neurons, and activation functions, are determined based on experimentation and validation performance.

3. Training

- The prepared data is divided into training and validation sets. The model is trained on the training set, which consists of input movie reviews and their corresponding sentiment labels.
- During training, the model's parameters (weights and biases) are adjusted using optimization algorithms, such as stochastic gradient descent (SGD) or Adam, to minimize a defined loss function (e.g., binary cross-entropy).
- The model iteratively processes batches of reviews, updating the parameters based on the gradient of the loss function, until convergence or a specified number of epochs.
- The performance of the model on the validation set is monitored to ensure it is not overfitting, and techniques like early stopping can be applied to prevent excessive training.

4. Evaluation

- The trained model is evaluated on a separate test set that was not seen during training or validation.
- Performance metrics such as accuracy, precision, recall, and F1 score are calculated to assess the model's classification performance.
- Additional evaluation measures like the confusion matrix and ROC curve may also be analyzed to gain insights into the model's predictions.

5. Prediction and Deployment

- Once the model has been trained and evaluated, it can be used to predict the sentiment of new, unseen movie reviews.
- The model is deployed in a production environment, where it takes input in the form of text and outputs the predicted sentiment (positive or negative).
- The deployment can be through an API or an interface that allows users to interact with the model and obtain sentiment predictions for movie reviews.

By iterative training on labeled data, adjusting parameters, and optimizing the model's ability to recognize patterns and sentiment in text, deep neural networks can effectively classify movie reviews into positive or negative categories.

Implementation:

```
1 from keras.datasets import imdb
```

```
1 (train_data, train_labels), (test_data, test_labels) = imdb.load_data(num_words=10000)
```

```
1 train_data[0][0]
```

1

```
1 # word_index is a dictionary mapping words to an integer index
2 word_index = imdb.get_word_index()
3 # We reverse it, mapping integer indices to words
4 reverse_word_index = dict([(value, key) for (key, value) in word_index.items()])
5 # We decode the review; note that our indices were offset by 3
6 # because 0, 1 and 2 are reserved indices for "padding", "start of sequence", and "unknown".
7 decoded_review = ' '.join([reverse_word_index.get(i - 3, '.') for i in train_data[0]])
```

```
1 decoded_review
```

". this film was just brilliant casting location scenery story direction everyone's really suited the part they played and you could just imagine being there robert . is an amazing actor and now the same being director . father came from the same scottish island as myself so i loved the fact there was a real connection with this film the witty remarks throughout the film were great it was just brilliant so much that i bought the film as soon as it was released for . and would recommend it to everyone to watch and the fly fishing was amazing really cried at the end it was so sad and you know what they say if you cry at a film it must have been good and this definitely was also . to the two little boy's that played the . of norman and paul they were just brilliant children are often left out of the . list i think because the stars that play them all grown up are such a big profile for the whole film but these children are amazing and should be praised for what they have done don't you think the whole story was so lovely because it was true and was someone's life after all that was shared with us all"

```
1 len(train_data)
```

25000

```
1 len(test_data)
```

25000

```
1 len(train_labels)
```

25000

```
1 len(test_labels)
```

25000

```
1 enumerate(train_data)
```

<enumerate at 0x78abec89fa80>

```
1 import numpy as np
2
3 def vectorize_sequences(sequences, dimension=10000):
4     # Create an all-zero matrix of shape (len(sequences), dimension)
5     results = np.zeros((len(sequences), dimension))
6     for i, sequence in enumerate(sequences):
7         results[i, sequence] = 1. # set specific indices of results[i] to 1s
8     return results
9
```

```

10 # Our vectorized training data
11 x_train = vectorize_sequences(train_data)
12 # Our vectorized test data
13 x_test = vectorize_sequences(test_data)

1 x_train[0]

array([0., 1., 1., ..., 0., 0., 0.])

1 # Our vectorized labels
2 y_train = np.asarray(train_labels).astype('float32')
3 y_test = np.asarray(test_labels).astype('float32')

1 x_train.shape

(25000, 10000)

1 len(x_train)

25000

1 type(x_train)

numpy.ndarray

1 x_val = x_train[:12000]
2 partial_x_train = x_train[12000:]
3
4 y_val = y_train[:12000]

5 partial_y_train = y_train[12000:]

1 from keras import models
2 from keras import layers

1 model = models.Sequential()
2 model.add(layers.Dense(16, activation='relu', input_shape=(10000,)))
3 model.add(layers.Dense(16, activation='relu'))
4 model.add(layers.Dense(1, activation='sigmoid'))
5
6 model.compile(optimizer='rmsprop',
7               loss='binary_crossentropy',
8               metrics=['accuracy'])
9
10 history=model.fit(x_train, y_train, epochs=10, batch_size=512, validation_data=(x_val, y_val))
11 results = model.evaluate(x_test, y_test)

Epoch 1/10
49/49 [=====] - 5s 84ms/step - loss: 0.4676 - accuracy: 0.8094 - val_loss: 0.3015 - val_accuracy: 0.9040
Epoch 2/10
49/49 [=====] - 2s 43ms/step - loss: 0.2683 - accuracy: 0.9040 - val_loss: 0.2074 - val_accuracy: 0.9317
Epoch 3/10
49/49 [=====] - 2s 43ms/step - loss: 0.2108 - accuracy: 0.9246 - val_loss: 0.1692 - val_accuracy: 0.9440
Epoch 4/10
49/49 [=====] - 2s 38ms/step - loss: 0.1808 - accuracy: 0.9339 - val_loss: 0.1429 - val_accuracy: 0.9555
Epoch 5/10
49/49 [=====] - 2s 38ms/step - loss: 0.1574 - accuracy: 0.9436 - val_loss: 0.1254 - val accuracy: 0.96

```

```

73
Epoch 7/10
49/49 [=====] - 3s 54ms/step - loss: 0.1257 - accuracy: 0.9563 - val_loss: 0.0979 - val_accuracy: 0.97
13
Epoch 8/10
49/49 [=====] - 2s 44ms/step - loss: 0.1152 - accuracy: 0.9614 - val_loss: 0.0901 - val_accuracy: 0.97
43
Epoch 9/10
49/49 [=====] - 2s 38ms/step - loss: 0.1035 - accuracy: 0.9644 - val_loss: 0.1150 - val_accuracy: 0.95
57
Epoch 10/10
49/49 [=====] - 2s 38ms/step - loss: 0.0929 - accuracy: 0.9694 - val_loss: 0.0697 - val_accuracy: 0.98
21
782/782 [=====] - 2s 2ms/step - loss: 0.3964 - accuracy: 0.8698

```

```

: 1 history_dict = history.history
  2 history_dict.keys()

: dict_keys(['loss', 'accuracy', 'val_loss', 'val_accuracy'])

```

```

: 1 print(history.history.keys())

dict_keys(['loss', 'accuracy', 'val_loss', 'val_accuracy'])

```

```

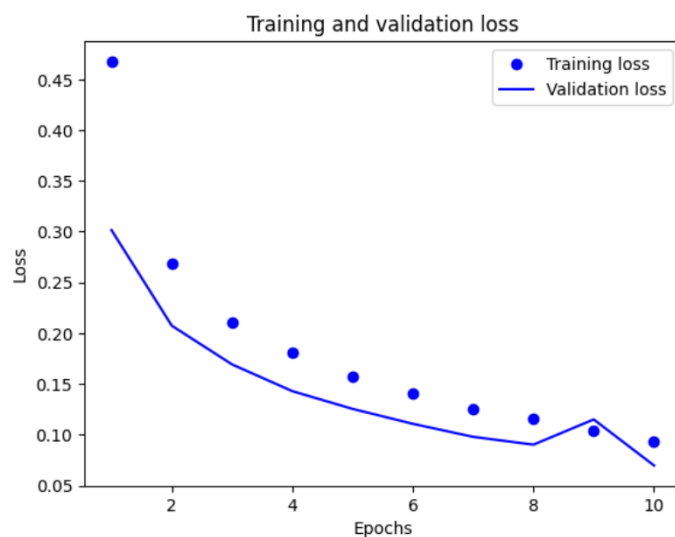
: 1 import matplotlib.pyplot as plt
  2
  3 acc = history.history['accuracy']
  4 val_acc = history.history['val_accuracy']
  5 loss = history.history['loss']
  6 val_loss = history.history['val_loss']
  7

```

```

7
8 epochs = range(1, len(acc) + 1)
9
10 # "bo" is for "blue dot"
11 plt.plot(epochs, loss, 'bo', label='Training loss')
12 # b is for "solid blue line"
13 plt.plot(epochs, val_loss, 'b', label='Validation loss')
14 plt.title('Training and validation loss')
15 plt.xlabel('Epochs')
16 plt.ylabel('Loss')
17 plt.legend()
18
19 plt.show()

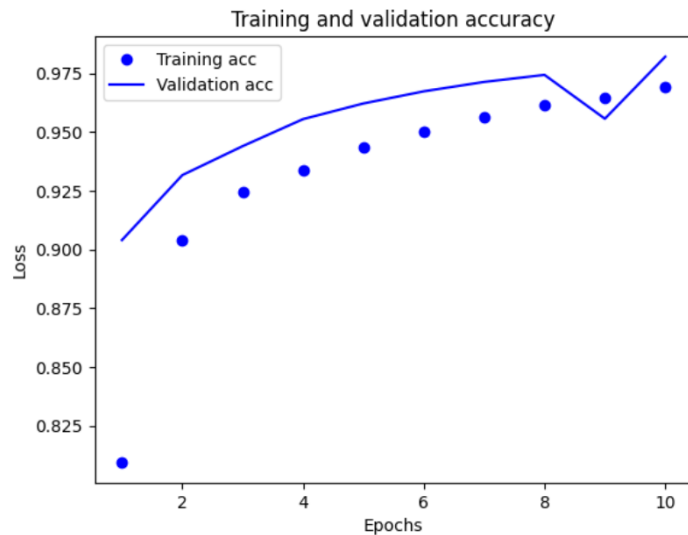
```



```

1 plt.clf() # clear figure
2 acc_values = history_dict['accuracy']
3 val_acc_values = history_dict['val_accuracy']
4
5 plt.plot(epochs, acc, 'bo', label='Training acc')
6 plt.plot(epochs, val_acc, 'b', label='Validation acc')
7 plt.title('Training and validation accuracy')
8 plt.xlabel('Epochs')
9 plt.ylabel('Loss')
10 plt.legend()
11
12 plt.show()

```



Conclusion:

The classification of movie reviews using deep neural networks and sentiment analysis techniques is a powerful approach for automating sentiment analysis tasks. By leveraging the capabilities of deep learning models and natural language processing techniques, we can develop models that accurately classify movie reviews as positive or negative based solely on their textual content.

Overall, the combination of deep neural networks, sentiment analysis techniques, and natural language processing enables efficient and accurate classification of movie reviews. This approach revolutionizes the way we can analyze and understand sentiment in textual data, opening doors to more effective decision-making and personalized user experiences in various domains.

Experiment No - 03

Title: Convolutional neural network: Use any dataset of plant disease and design a plant disease detection system using CNN.

Objective: Software Requirement: Students should be able to implement a Convolutional Neural Network. Design a plant disease detection system using CNN.

Software Requirement:

Operating system: Windows, Linux, MacOS

Programming Language: Python, C++, Java.

Theoretical Concept:

What Is a Convolutional Neural Network?

A Convolutional Neural Network (CNN) is a type of deep learning algorithm that is primarily used for image recognition and processing, but it can also be applied to other types of data such as sound and text. CNNs are inspired by the biological processes in the visual cortex of the human brain.

The key components of a CNN include:

Convolutional Layers: These layers apply convolution operations to the input data using filters or kernels. Convolution involves sliding a filter over the input image, performing element-wise multiplication with the pixels in the filter and corresponding region of the input, and then summing up the results to produce a feature map.

Pooling Layers: Pooling layers downsample the feature maps produced by the convolutional layers. Common pooling operations include max pooling and average pooling, which reduce the dimensionality of the feature maps while retaining important information.

Activation Functions: Activation functions introduce non-linearity to the network, allowing it to learn complex patterns in the data. Common activation functions used in CNNs include ReLU (Rectified Linear Unit), sigmoid, and tanh.

Fully Connected Layers: After several convolutional and pooling layers, the high-level reasoning in the neural network is done via fully connected layers. These layers connect every neuron in one layer to every neuron in the next layer, allowing for complex relationships to be learned.

Output Layer: The output layer produces the final prediction or classification. The number of neurons in this layer depends on the problem at hand. For example, in image classification tasks, the number of neurons in the output layer corresponds to the number of classes.

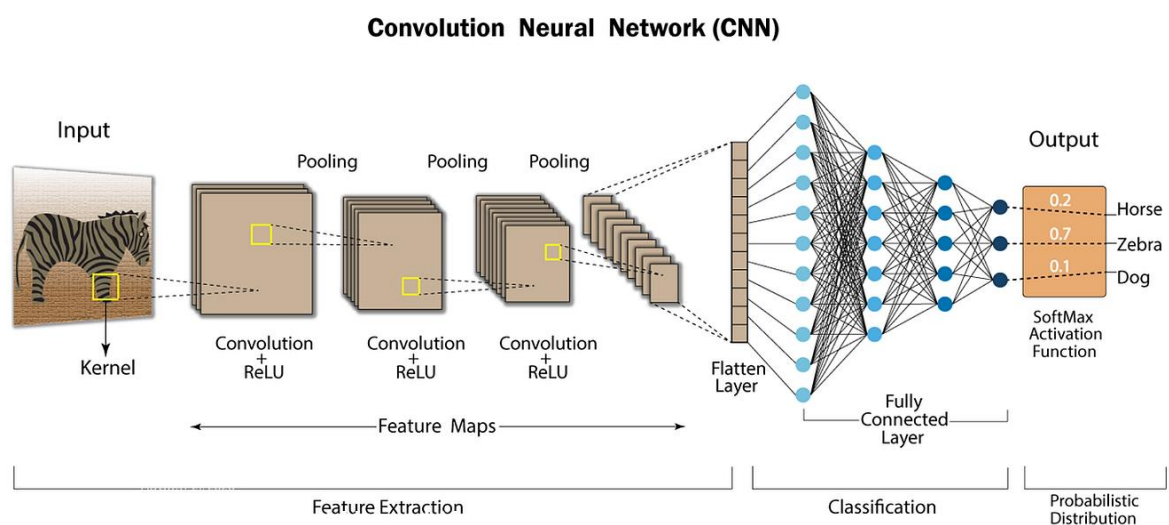


Figure: Simple Architecture of CNN

Plant Disease Detection System

A plant disease detection system using Convolutional Neural Networks (CNNs) typically involves the following steps:

Data Collection: Gather a dataset consisting of images of healthy plants as well as plants affected by various diseases. These images should cover different plant species, diseases, and stages of disease progression.

Data Preprocessing: Preprocess the images to ensure uniformity in size, color, and orientation. Common preprocessing steps include resizing, normalization, and augmentation to increase the diversity of the dataset.

Model Training: Train a CNN model using the preprocessed dataset. The CNN architecture will consist of convolutional layers, pooling layers, activation functions, and fully connected layers. The goal is to teach the model to distinguish between healthy and diseased plants based on the visual patterns present in the images.

Validation: Validate the trained model using a separate dataset or through techniques like cross-validation to assess its performance and generalization ability.

Deployment: Deploy the trained model into a system or application where it can be used for disease detection. This could be a mobile app, a web-based tool, or an embedded system that analyzes images captured from a camera.

Real-time Detection: Implement the system to allow real-time detection of plant diseases. This may involve capturing images of plants using cameras or smartphones, preprocessing the images, feeding them into the trained model, and then providing feedback on the presence of diseases.

Feedback Loop: Continuously improve the system by collecting feedback on its performance and updating the model accordingly. This could involve retraining the model with new data, fine-tuning hyperparameters, or adjusting the architecture to improve accuracy and robustness.

Implementation:

import dataset

```
1 !pip install kaggle
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.10/dist-packages (1.5.16)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.10/dist-packages (from kaggle) (1.16.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.10/dist-packages (from kaggle) (2024.2.2)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.10/dist-packages (from kaggle) (2.8.2)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from kaggle) (2.31.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from kaggle) (4.66.2)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.10/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.10/dist-packages (from kaggle) (2.0.7)
Requirement already satisfied: bleach in /usr/local/lib/python3.10/dist-packages (from kaggle) (6.1.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.10/dist-packages (from bleach->kaggle) (0.5.1)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.10/dist-packages (from python-slugify->kaggle) (1.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->kaggle) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->kaggle) (3.6)
```

```
1 from google.colab import files
2 files.upload()
```

[Choose Files](#) No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving kaggle.json to kaggle (1).json

```
{'kaggle (1).json': b'{"username": "simmimaster", "key": "66aafa3345790b2104fb1ec3386044cb"}'}
```



```
mkdir: cannot create directory '/root/.kaggle': File exists
```

```
1 cp /content/kaggle.json ~/.kaggle/
```

```
1 !chmod 600 ~/.kaggle/kaggle.json
```

```
1 !kaggle datasets download -d mohitsingh1804/plantvillage
```

```
plantvillage.zip: Skipping, found more recently modified local copy (use --force to force download)
```

```
1 !unzip plantvillage.zip
```

inflating:	PlantVillage\val\Soybean	healthy\53168d1c-ac90-4039-9eff-7479475a6b74	RS_HL	6355.JPG
inflating:	PlantVillage\val\Soybean	healthy\534ab094-bbbe-4a2c-9345-d93dd46e9b4	RS_HL	4693.JPG
inflating:	PlantVillage\val\Soybean	healthy\5361ba4f-351e-466c-9935-884f535c00df	RS_HL	2890.JPG
inflating:	PlantVillage\val\Soybean	healthy\537e5409-d824-4859-967f-3c502f13deec	RS_HL	5474.JPG
inflating:	PlantVillage\val\Soybean	healthy\538964da-545b-4bce-bc1d-0932898d45e6	RS_HL	5230.JPG
inflating:	PlantVillage\val\Soybean	healthy\538ccfe2-1509-40d7-beae-aef7c84e1e5d	RS_HL	3717.JPG
inflating:	PlantVillage\val\Soybean	healthy\5397b6fa-aeeb-4f1c-b554-bdb604db23c3	RS_HL	6508.JPG
inflating:	PlantVillage\val\Soybean	healthy\53dc0466-621c-43e1-9c5e-3c40f3d50171	RS_HL	6381.JPG
inflating:	PlantVillage\val\Soybean	healthy\53e3fcc2-3234-4330-b551-083c72fb6e12	RS_HL	6982.JPG
inflating:	PlantVillage\val\Soybean	healthy\5461083c-0075-4b4e-8899-47e0771a393e	RS_HL	7460.JPG
inflating:	PlantVillage\val\Soybean	healthy\5510e8d6-d957-42f9-b880-6231c4b2c43f	RS_HL	3535.JPG
inflating:	PlantVillage\val\Soybean	healthy\551614f2-d150-4fed-a435-cdb9f5a5b90c	RS_HL	4065.JPG
inflating:	PlantVillage\val\Soybean	healthy\55ed6bf1-9baf-45d9-4b4b-4f19c7dc271a	RS_HL	7312.JPG
inflating:	PlantVillage\val\Soybean	healthy\561d3eaf-4ff0-4580-9958-55949f3a88b	RS_HL	6420.JPG

```
import library
```

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import tensorflow as tf
4 from tensorflow.keras.models import Sequential
5 from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
6 from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
1 train_datagen = ImageDataGenerator(rescale=1./255, shear_range=0.2, zoom_range=0.2, horizontal_flip=True)
2 test_datagen = ImageDataGenerator(rescale=1./255)
3
4 train_set = train_datagen.flow_from_directory('/content/PlantVillage/train', target_size=(64, 64), batch_size=32, class_mode='categorical_crossentropy')
5 test_set = test_datagen.flow_from_directory('/content/PlantVillage/val', target_size=(64, 64), batch_size=32, class_mode='categorical_crossentropy')
```

```
Found 43444 images belonging to 38 classes.  
Found 10861 images belonging to 38 classes.
```

```
1 # Number of classes
2 num_classes = len(train_set.class_indices)
3 print("Number of classes:", num_classes)
4
5 # Names of classes
6 class_names = list(train_set.class_indices.keys())
7 print("Class names:", class_names)
```

Number of classes: 38
Class names: ['Apple__Apple_scab', 'Apple__Black_rot', 'Apple__Cedar_apple_rust', 'Apple__healthy', 'Blueberry__healthy', 'Cherry (including sour)__Powdery_mildew', 'Cherry (including sour)__healthy', 'Corn (maize)__Cercospora leaf spot Gray leaf

```

_spot', 'Corn_(maize)___Common_rust_', 'Corn_(maize)___Northern_Leaf_Blight', 'Corn_(maize)___healthy', 'Grape___Black_rot', 'G
rape___Esca_(Black_Measles)', 'Grape___Leaf_blight_(Isariopsis_Leaf_Spot)', 'Grape___healthy', 'Orange___Haunglongbing_(Citrus
greening)', 'Peach___Bacterial_spot', 'Peach___healthy', 'Pepper_bell___Bacterial_spot', 'Pepper_bell___healthy', 'Potato___E
arly_blight', 'Potato___Late_blight', 'Potato___healthy', 'Raspberry___healthy', 'Soybean___healthy', 'Squash___Powdery_milde
w', 'Strawberry___Leaf_scorch', 'Strawberry___healthy', 'Tomato___Bacterial_spot', 'Tomato___Early_blight', 'Tomato___Late_blig
ht', 'Tomato___Leaf_Mold', 'Tomato___Septoria_leaf_spot', 'Tomato___Spider_mites Two-spotted_spider_mite', 'Tomato___Target_Spo
t', 'Tomato___Tomato_Yellow_Leaf_Curl_Virus', 'Tomato___Tomato_mosaic_virus', 'Tomato___healthy']

```

```

1 train_datagen = ImageDataGenerator(rescale=1./255, shear_range=0.2, zoom_range=0.2, horizontal_flip=True)
2 test_datagen = ImageDataGenerator(rescale=1./255)
3
4 train_set = train_datagen.flow_from_directory('/content/PlantVillage/train', target_size=(64, 64), batch_size=32, class_mode='ca
5 test_set = test_datagen.flow_from_directory('/content/PlantVillage/val', target_size=(64, 64), batch_size=32, class_mode='ca

```

Found 1000 images belonging to 2 classes.
Found 251 images belonging to 2 classes.

Build the CNN Model:

```

1 model = Sequential()
2
3 model.add(Conv2D(32, (3, 3), input_shape=(64, 64, 3), activation='relu'))
4 model.add(MaxPooling2D(pool_size=(2, 2)))
5
6 model.add(Conv2D(64, (3, 3), activation='relu'))
7 model.add(MaxPooling2D(pool_size=(2, 2)))
8
9 model.add(Conv2D(128, (3, 3), activation='relu'))
10
11 model.add(MaxPooling2D(pool_size=(2, 2)))
12 model.add(Flatten())
13
14 model.add(Dense(units=128, activation='relu'))
15 model.add(Dropout(0.5))
16
17 model.add(Dense(units=2, activation='softmax')) # Change units to 2 for two classes
18

```

Compile the Model:

```

1 model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

```

Train the Model:

```

1 history = model.fit(train_set, steps_per_epoch=len(train_set), epochs=30, validation_data=test_set, validation_steps=len(test

```

```

Epoch 1/30
32/32 [=====] - 12s 322ms/step - loss: 0.6602 - accuracy: 0.5960 - val_loss: 0.5261 - val_accuracy: 0.
7012
Epoch 2/30
32/32 [=====] - 10s 297ms/step - loss: 0.3187 - accuracy: 0.8810 - val_loss: 0.3198 - val_accuracy: 0.
8765
Epoch 3/30
32/32 [=====] - 9s 274ms/step - loss: 0.2088 - accuracy: 0.9250 - val_loss: 0.2486 - val_accuracy: 0.9
163

```

```
Epoch 4/30
32/32 [=====] - 9s 276ms/step - loss: 0.1691 - accuracy: 0.9330 - val_loss: 0.2614 - val_accuracy: 0.8964
Epoch 5/30
32/32 [=====] - 10s 299ms/step - loss: 0.1577 - accuracy: 0.9380 - val_loss: 0.2777 - val_accuracy: 0.9084
Epoch 6/30
32/32 [=====] - 9s 292ms/step - loss: 0.1501 - accuracy: 0.9580 - val_loss: 0.1895 - val_accuracy: 0.9283
Epoch 7/30
32/32 [=====] - 8s 253ms/step - loss: 0.1383 - accuracy: 0.9520 - val_loss: 0.2662 - val_accuracy: 0.9124
Epoch 8/30
32/32 [=====] - 8s 242ms/step - loss: 0.1253 - accuracy: 0.9510 - val_loss: 0.2366 - val_accuracy: 0.9124
Epoch 9/30
32/32 [=====] - 15s 472ms/step - loss: 0.1049 - accuracy: 0.9610 - val_loss: 0.1919 - val_accuracy: 0.9243
Epoch 10/30
32/32 [=====] - 10s 296ms/step - loss: 0.1066 - accuracy: 0.9650 - val_loss: 0.3468 - val_accuracy: 0.8884
Epoch 11/30
32/32 [=====] - 9s 279ms/step - loss: 0.0948 - accuracy: 0.9650 - val_loss: 0.1288 - val_accuracy: 0.9522
Epoch 12/30
32/32 [=====] - 9s 286ms/step - loss: 0.1464 - accuracy: 0.9420 - val_loss: 0.1906 - val_accuracy: 0.9084
Epoch 13/30
32/32 [=====] - 10s 297ms/step - loss: 0.0955 - accuracy: 0.9620 - val_loss: 0.2656 - val_accuracy: 0.8924
Epoch 14/30
32/32 [=====] - 9s 287ms/step - loss: 0.0874 - accuracy: 0.9720 - val_loss: 0.2628 - val_accuracy: 0.8924
Epoch 15/30
32/32 [=====] - 8s 254ms/step - loss: 0.0747 - accuracy: 0.9720 - val_loss: 0.1692 - val_accuracy: 0.9323
Epoch 16/30
32/32 [=====] - 9s 291ms/step - loss: 0.0831 - accuracy: 0.9670 - val_loss: 0.0864 - val_accuracy: 0.9641
Epoch 17/30
32/32 [=====] - 10s 299ms/step - loss: 0.0719 - accuracy: 0.9760 - val_loss: 0.0971 - val_accuracy: 0.9522
Epoch 18/30
32/32 [=====] - 9s 266ms/step - loss: 0.0722 - accuracy: 0.9740 - val_loss: 0.1671 - val_accuracy: 0.9283
Epoch 19/30
32/32 [=====] - 8s 260ms/step - loss: 0.0628 - accuracy: 0.9760 - val_loss: 0.0826 - val_accuracy: 0.9721
Epoch 20/30
32/32 [=====] - 10s 299ms/step - loss: 0.0650 - accuracy: 0.9810 - val_loss: 0.0839 - val_accuracy: 0.9721
Epoch 21/30
32/32 [=====] - 10s 302ms/step - loss: 0.0577 - accuracy: 0.9770 - val_loss: 0.8295 - val_accuracy: 0.7769
Epoch 22/30
32/32 [=====] - 9s 286ms/step - loss: 0.0548 - accuracy: 0.9790 - val_loss: 0.1454 - val_accuracy: 0.9402
Epoch 23/30
32/32 [=====] - 11s 334ms/step - loss: 0.0670 - accuracy: 0.9710 - val_loss: 0.2174 - val_accuracy: 0.9044
Epoch 24/30
32/32 [=====] - 10s 298ms/step - loss: 0.0496 - accuracy: 0.9820 - val_loss: 0.1506 - val_accuracy: 0.
```

```

32/32 [=====] - 10s 298ms/step - loss: 0.0496 - accuracy: 0.9820 - val_loss: 0.1506 - val_accuracy: 0.9363
Epoch 25/30
32/32 [=====] - 10s 305ms/step - loss: 0.0358 - accuracy: 0.9880 - val_loss: 0.0598 - val_accuracy: 0.9841
Epoch 26/30
32/32 [=====] - 8s 253ms/step - loss: 0.0433 - accuracy: 0.9850 - val_loss: 0.1800 - val_accuracy: 0.9203
Epoch 27/30
32/32 [=====] - 8s 251ms/step - loss: 0.0314 - accuracy: 0.9890 - val_loss: 0.1489 - val_accuracy: 0.9363
Epoch 28/30
32/32 [=====] - 9s 278ms/step - loss: 0.0420 - accuracy: 0.9850 - val_loss: 0.1207 - val_accuracy: 0.9602
Epoch 29/30
32/32 [=====] - 10s 302ms/step - loss: 0.0302 - accuracy: 0.9880 - val_loss: 0.0893 - val_accuracy: 0.9562
Epoch 30/30
32/32 [=====] - 9s 293ms/step - loss: 0.0229 - accuracy: 0.9910 - val_loss: 0.0354 - val_accuracy: 0.9880

```

Evaluate the Model:

```

: 1 test_loss, test_acc = model.evaluate(test_set, steps=len(test_set))
  2 print("Test Accuracy:", test_acc)

```

```

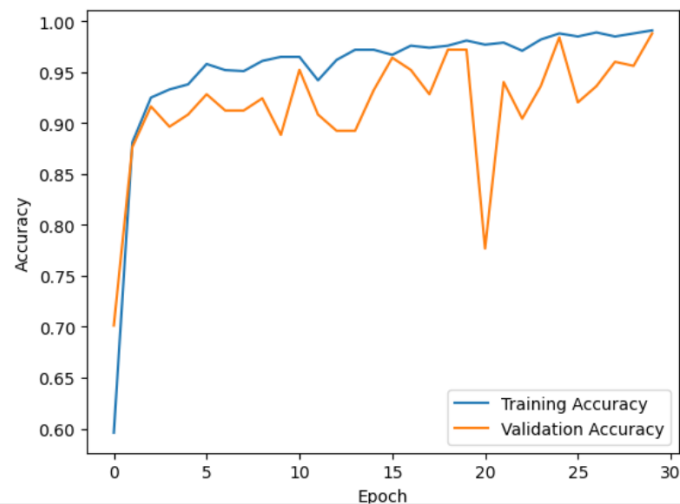
8/8 [=====] - 1s 71ms/step - loss: 0.0354 - accuracy: 0.9880
Test Accuracy: 0.9880478382110596

```

```

1 plt.plot(history.history['accuracy'], label='Training Accuracy')
2 plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
3 plt.xlabel('Epoch')
4 plt.ylabel('Accuracy')
5 plt.legend()
6 plt.show()

```



PRE-TRAIN_CNN model

Import necessary libraries:

```
: 1 import numpy as np
2 import matplotlib.pyplot as plt
3 import tensorflow as tf
4 from tensorflow.keras.applications import VGG16
5 from tensorflow.keras.models import Sequential
6 from tensorflow.keras.layers import Flatten, Dense, Dropout
7 from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

Load the pre-trained VGG16 model ¶

```
: 1 # Load the pre-trained VGG16 model (excluding the top classification layer)
2 base_model = VGG16(weights='imagenet', include_top=False, input_shape=(64, 64, 3))
```

pre-trained model

```
: 1 # Freeze the layers of the pre-trained model
2 for layer in base_model.layers:
3     layer.trainable = False
4
5 # Create a new model on top of the pre-trained base model
6 model = Sequential()
7
8 model.add(base_model)
9 model.add(Flatten())
10 model.add(Dense(units=128, activation='relu'))
11 model.add(Dropout(0.5))
12 model.add(Dense(units=2, activation='softmax')) # Change units to 2 for two classes
```

Compile the model

```
: 1 # Compile the model
2 model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Data preprocessing

```
: 1 # Data preprocessing
2 train_datagen = ImageDataGenerator(rescale=1./255, shear_range=0.2, zoom_range=0.2, horizontal_flip=True)
3 test_datagen = ImageDataGenerator(rescale=1./255)
4
5 train_set = train_datagen.flow_from_directory('/content/PlantVillage/train', target_size=(64, 64), batch_size=32, class_mode='categorical')
6 test_set = test_datagen.flow_from_directory('/content/PlantVillage/val', target_size=(64, 64), batch_size=32, class_mode='categorical')
```

Found 1000 images belonging to 2 classes.
Found 251 images belonging to 2 classes.

Train the model

```

1 # Train the model
2 history = model.fit(train_set, steps_per_epoch=len(train_set), epochs=25, validation_data=test_set, validation_steps=len(test_set))

```

Epoch 1/25
32/32 [=====] - 68s 2s/step - loss: 0.3257 - accuracy: 0.8630 - val_loss: 0.1516 - val_accuracy: 0.9482

Epoch 2/25
32/32 [=====] - 57s 2s/step - loss: 0.1272 - accuracy: 0.9500 - val_loss: 0.1256 - val_accuracy: 0.9482

Epoch 3/25
32/32 [=====] - 65s 2s/step - loss: 0.1023 - accuracy: 0.9600 - val_loss: 0.1193 - val_accuracy: 0.9522

Epoch 4/25
32/32 [=====] - 66s 2s/step - loss: 0.1001 - accuracy: 0.9630 - val_loss: 0.2078 - val_accuracy: 0.9124

Epoch 5/25
32/32 [=====] - 70s 2s/step - loss: 0.0896 - accuracy: 0.9630 - val_loss: 0.0924 - val_accuracy: 0.9602

Epoch 6/25
32/32 [=====] - 67s 2s/step - loss: 0.0647 - accuracy: 0.9740 - val_loss: 0.1014 - val_accuracy: 0.9602

Epoch 7/25
32/32 [=====] - 56s 2s/step - loss: 0.0690 - accuracy: 0.9720 - val_loss: 0.0884 - val_accuracy: 0.9602

Epoch 8/25
32/32 [=====] - 66s 2s/step - loss: 0.0663 - accuracy: 0.9740 - val_loss: 0.0930 - val_accuracy: 0.9641

Epoch 9/25
32/32 [=====] - 65s 2s/step - loss: 0.0714 - accuracy: 0.9670 - val_loss: 0.0846 - val_accuracy: 0.9681

Epoch 9/25
32/32 [=====] - 65s 2s/step - loss: 0.0714 - accuracy: 0.9670 - val_loss: 0.0846 - val_accuracy: 0.9681

Epoch 10/25
32/32 [=====] - 55s 2s/step - loss: 0.0614 - accuracy: 0.9740 - val_loss: 0.1526 - val_accuracy: 0.9402

Epoch 11/25
32/32 [=====] - 58s 2s/step - loss: 0.0624 - accuracy: 0.9750 - val_loss: 0.0973 - val_accuracy: 0.9562

Epoch 12/25
32/32 [=====] - 56s 2s/step - loss: 0.0525 - accuracy: 0.9800 - val_loss: 0.0782 - val_accuracy: 0.9721

Epoch 13/25
32/32 [=====] - 65s 2s/step - loss: 0.0629 - accuracy: 0.9770 - val_loss: 0.0765 - val_accuracy: 0.9681

Epoch 14/25
32/32 [=====] - 66s 2s/step - loss: 0.0525 - accuracy: 0.9860 - val_loss: 0.0846 - val_accuracy: 0.9602

Epoch 15/25
32/32 [=====] - 65s 2s/step - loss: 0.0547 - accuracy: 0.9790 - val_loss: 0.0855 - val_accuracy: 0.9562

Epoch 16/25
32/32 [=====] - 55s 2s/step - loss: 0.0460 - accuracy: 0.9840 - val_loss: 0.0888 - val_accuracy: 0.9522

Epoch 17/25
32/32 [=====] - 65s 2s/step - loss: 0.0692 - accuracy: 0.9730 - val_loss: 0.0764 - val_accuracy: 0.9681

Epoch 18/25
32/32 [=====] - 66s 2s/step - loss: 0.0587 - accuracy: 0.9800 - val_loss: 0.1739 - val_accuracy: 0.9323

Epoch 19/25

```

32/32 [=====] - 56s 2s/step - loss: 0.0546 - accuracy: 0.9760 - val_loss: 0.0812 - val_accuracy: 0.976
1
Epoch 20/25
32/32 [=====] - 66s 2s/step - loss: 0.0547 - accuracy: 0.9800 - val_loss: 0.0748 - val_accuracy: 0.956
2
Epoch 21/25
32/32 [=====] - 64s 2s/step - loss: 0.0450 - accuracy: 0.9830 - val_loss: 0.0838 - val_accuracy: 0.952
2
Epoch 22/25
32/32 [=====] - 66s 2s/step - loss: 0.0459 - accuracy: 0.9860 - val_loss: 0.0703 - val_accuracy: 0.964
1
Epoch 23/25
32/32 [=====] - 65s 2s/step - loss: 0.0451 - accuracy: 0.9830 - val_loss: 0.0982 - val_accuracy: 0.952
2
Epoch 24/25
32/32 [=====] - 67s 2s/step - loss: 0.0417 - accuracy: 0.9820 - val_loss: 0.0771 - val_accuracy: 0.964
1
Epoch 25/25
32/32 [=====] - 64s 2s/step - loss: 0.0335 - accuracy: 0.9880 - val_loss: 0.0954 - val_accuracy: 0.952
2

```

Evaluate the model

```

1 # Evaluate the model
2 test_loss, test_acc = model.evaluate(test_set, steps=len(test_set))
3 print("Test Accuracy:", test_acc)

```

```

8/8 [=====] - 12s 2s/step - loss: 0.0954 - accuracy: 0.9522
Test Accuracy: 0.9521912336349487

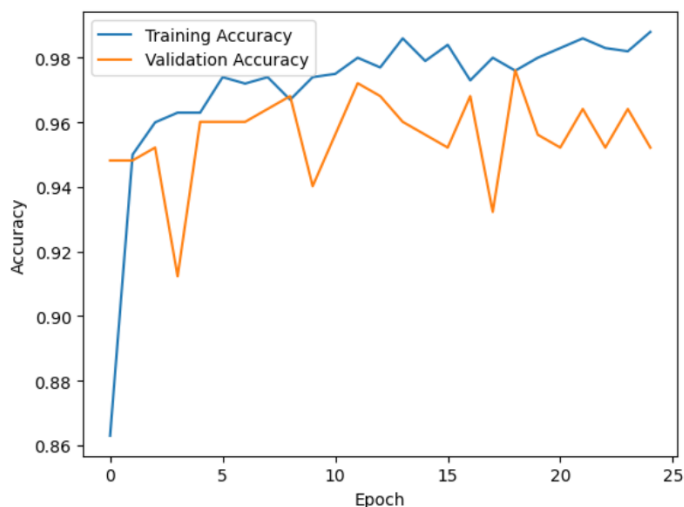
```

Plot training history

```

1 # Plot training history
2 plt.plot(history.history['accuracy'], label='Training Accuracy')
3 plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
4 plt.xlabel('Epoch')
5 plt.ylabel('Accuracy')
6 plt.legend()
7 plt.show()

```



Conclusion: In this way, we used a Convolutional neural network to design a plant disease detection system.

Experiment No - 04

Title: Recurrent neural network: Use the Google stock prices dataset and design a time series analysis and prediction system using RNN.

Objective: Software Requirement: Students should be able to implement a Recurrent Neural Network.

Design a time series analysis and prediction system using RNN.

Software Requirement:

Operating system: Windows, Linux, MacOS

Programming Language: Python, C++, Java.

Theoretical Concept:

What Is a Recurrent Neural Network?

A Recurrent Neural Network (RNN) is a class of artificial neural networks specifically designed to recognize patterns in sequences of data, such as time series, text, genomes, or spoken numerical data. Unlike traditional feedforward neural networks, RNNs have a "memory" that captures information about what has been calculated so far, allowing the network to use previous outputs as inputs for hidden states.

The key components of a RNN include:

Recurrent Layers: These layers contain loops, allowing information to persist. They process data one time step at a time, taking both the current input and the hidden state from the previous time step to generate a new hidden state and output.

Hidden State (Memory): Pooling This acts as the network's memory, holding information about previous data in the sequence. It is updated continuously as the network reads through the time steps.

Activation Functions: Like other networks, activation functions (often tanh or ReLU) are used to introduce non-linearity into the network, allowing it to model complex patterns in the sequential data.

Fully Connected (Dense) Output Layers: After the sequence is processed by the recurrent layers, the final hidden state (or sometimes all states) is passed to a dense layer to produce the final continuous numerical prediction (e.g., the stock price for the next day).

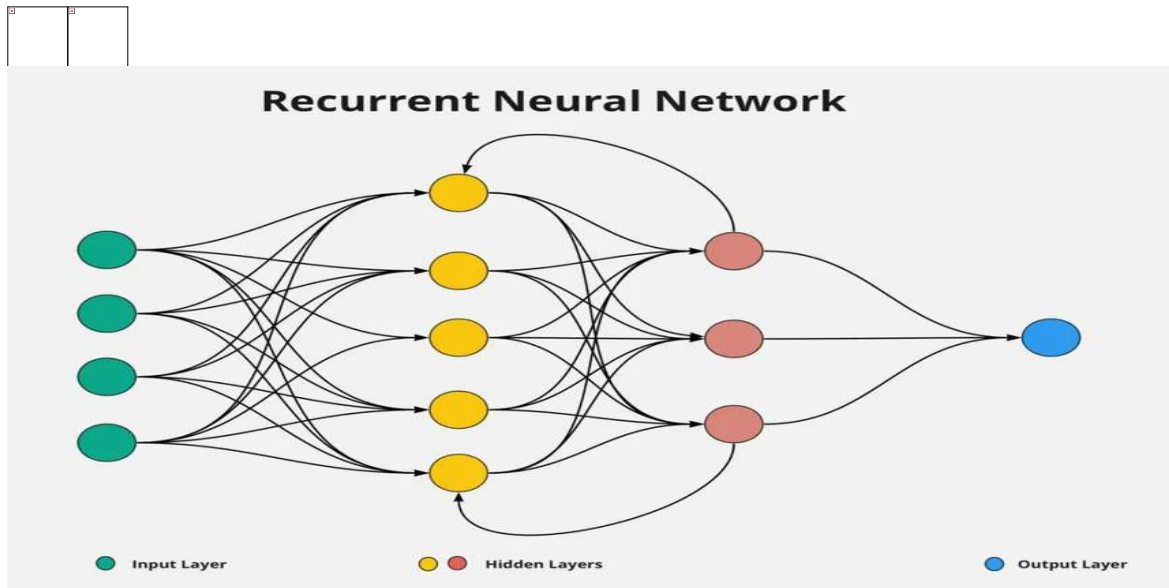


Figure: Simple Architecture of RNN

Time Series Analysis and Prediction System

A time series prediction system using Recurrent Neural Networks (RNNs) typically involves the following steps:

Data Collection: Gather a dataset consisting of historical stock prices. For this experiment, the Google Stock Prices dataset (containing Open, High, Low, Close, and Volume data) over a specific number of years will be used.

Data Preprocessing: Time series data must be scaled before feeding it to an RNN to ensure the model converges quickly and accurately. Common steps include using Min-Max Scaling to compress the data between 0 and 1, and creating time-step sequences (e.g., using the past 60 days of stock prices to predict the 61st day).

Model Training: Train an RNN model using the preprocessed sequential dataset. The architecture will consist of Recurrent layers (SimpleRNN, LSTM, or GRU), dropout layers to prevent overfitting, and a final fully connected dense layer to output the predicted stock value.

Validation: Validate the trained model using a chronologically separated test dataset to prevent data leakage. The model's predictions are compared against actual historical data to assess its performance.

Deployment: Deploy the trained model into a financial analysis tool or dashboard where it can be used for forecasting future trends.

Real-time Detection: Implement the system to fetch live market data, scale it using the saved scaler, and feed it into the model to predict upcoming price movements.

Feedback Loop: Continuously improve the system by retraining the model periodically as new market data becomes available to adjust to changing market conditions.

Implementation:

```
# Import necessary Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, SimpleRNN, Dropout

# Load the dataset (Assuming CSV is downloaded from Kaggle/Yahoo Finance)
dataset_train = pd.read_csv('Google_Stock_Price_Train.csv')
training_set = dataset_train.iloc[:, 1:2].values # Selecting the 'Open' price column

# Feature Scaling
sc = MinMaxScaler(feature_range=(0, 1))
training_set_scaled = sc.fit_transform(training_set)

# Creating a data structure with 60 time steps and 1 output
X_train = []
y_train = []
for i in range(60, len(training_set_scaled)):
    X_train.append(training_set_scaled[i-60:i, 0])
    y_train.append(training_set_scaled[i, 0])
X_train, y_train = np.array(X_train), np.array(y_train)

# Reshaping data for RNN input (samples, time steps, features)
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))

print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
```

```

# Build the RNN Model
model = Sequential()

# Adding the first RNN Layer and some Dropout regularisation
model.add(SimpleRNN(units=50, activation='tanh', return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(Dropout(0.2))

# Adding a second RNN Layer and some Dropout regularisation
model.add(SimpleRNN(units=50, activation='tanh', return_sequences=True))
model.add(Dropout(0.2))

# Adding a third RNN Layer and some Dropout regularisation
model.add(SimpleRNN(units=50, activation='tanh', return_sequences=False))
model.add(Dropout(0.2))

# Adding the output Layer
model.add(Dense(units=1))

# Compile the Model
model.compile(optimizer='adam', loss='mean_squared_error')

# Train the Model
history = model.fit(X_train, y_train, epochs=30, batch_size=32)

```

```

# Evaluate the Model / Predict on Test Data
dataset_test = pd.read_csv('Google_Stock_Price_Test.csv')
real_stock_price = dataset_test.iloc[:, 1:2].values

# Getting the predicted stock price
dataset_total = pd.concat((dataset_train['Open'], dataset_test['Open']), axis=0)
inputs = dataset_total[len(dataset_total) - len(dataset_test) - 60:].values
inputs = inputs.reshape(-1,1)
inputs = sc.transform(inputs)

X_test = []
for i in range(60, 80): # Assuming 20 test days
    X_test.append(inputs[i-60:i, 0])
X_test = np.array(X_test)
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

predicted_stock_price = model.predict(X_test)
predicted_stock_price = sc.inverse_transform(predicted_stock_price) # Un-scale values

# Plotting the results
plt.plot(real_stock_price, color='red', label='Real Google Stock Price')
plt.plot(predicted_stock_price, color='blue', label='Predicted Google Stock Price')
plt.title('Google Stock Price Prediction')
plt.xlabel('Time')
plt.ylabel('Google Stock Price')
plt.legend()
plt.show()

```

```

1 # Train the model
2 history = model.fit(train_set, steps_per_epoch=len(train_set), epochs=25, validation_data=test_set, validation_steps=len(test_set))

```

```

Epoch 1/25
32/32 [=====] - 68s 2s/step - loss: 0.3257 - accuracy: 0.8630 - val_loss: 0.1516 - val_accuracy: 0.948
2
Epoch 2/25
32/32 [=====] - 57s 2s/step - loss: 0.1272 - accuracy: 0.9500 - val_loss: 0.1256 - val_accuracy: 0.948
2
Epoch 3/25
32/32 [=====] - 65s 2s/step - loss: 0.1023 - accuracy: 0.9600 - val_loss: 0.1193 - val_accuracy: 0.952
2
Epoch 4/25
32/32 [=====] - 66s 2s/step - loss: 0.1001 - accuracy: 0.9630 - val_loss: 0.2078 - val_accuracy: 0.912
4
Epoch 5/25
32/32 [=====] - 70s 2s/step - loss: 0.0896 - accuracy: 0.9630 - val_loss: 0.0924 - val_accuracy: 0.960
2
Epoch 6/25
32/32 [=====] - 67s 2s/step - loss: 0.0647 - accuracy: 0.9740 - val_loss: 0.1014 - val_accuracy: 0.960
2
Epoch 7/25
32/32 [=====] - 56s 2s/step - loss: 0.0690 - accuracy: 0.9720 - val_loss: 0.0884 - val_accuracy: 0.960
2
Epoch 8/25
32/32 [=====] - 66s 2s/step - loss: 0.0663 - accuracy: 0.9740 - val_loss: 0.0930 - val_accuracy: 0.964
1
Epoch 9/25
32/32 [=====] - 65s 2s/step - loss: 0.0714 - accuracy: 0.9670 - val_loss: 0.0846 - val_accuracy: 0.968
1
32/32 [=====] - 56s 2s/step - loss: 0.0546 - accuracy: 0.9760 - val_loss: 0.0812 - val_accuracy: 0.976
1
Epoch 20/25
32/32 [=====] - 66s 2s/step - loss: 0.0547 - accuracy: 0.9800 - val_loss: 0.0748 - val_accuracy: 0.956
2
Epoch 21/25
32/32 [=====] - 64s 2s/step - loss: 0.0450 - accuracy: 0.9830 - val_loss: 0.0838 - val_accuracy: 0.952
2
Epoch 22/25
32/32 [=====] - 66s 2s/step - loss: 0.0459 - accuracy: 0.9860 - val_loss: 0.0703 - val_accuracy: 0.964
1
Epoch 23/25
32/32 [=====] - 65s 2s/step - loss: 0.0451 - accuracy: 0.9830 - val_loss: 0.0982 - val_accuracy: 0.952
2
Epoch 24/25
32/32 [=====] - 67s 2s/step - loss: 0.0417 - accuracy: 0.9820 - val_loss: 0.0771 - val_accuracy: 0.964
1
Epoch 25/25
32/32 [=====] - 64s 2s/step - loss: 0.0335 - accuracy: 0.9880 - val_loss: 0.0954 - val_accuracy: 0.952
2

```

Evaluate the model

```

1 # Evaluate the model
2 test_loss, test_acc = model.evaluate(test_set, steps=len(test_set))
3 print("Test Accuracy:", test_acc)

```

```

8/8 [=====] - 12s 2s/step - loss: 0.0954 - accuracy: 0.9522
Test Accuracy: 0.9521912336349487

```

Conclusion: In this way, we used a Recurrent neural network to design a time series analysis and prediction system to forecast Google stock prices based on historical trends